

Audit research notes

Matteo Franzil, Valentino Armani, Luis Augusto Dias Knob, Domenico Siracusa

June 17, 2024

Using ML models for learning templates

- Idea: use a ML model to generalize and attempt to match 'templates' of behaviour in the logs. For example: we already have an idea of how namespaces are created, deleted, and so on. We could use a model to learn these templates and then match them in the logs. If no match is found, then an attempt could be made to generate a new template based on the most similar ones.

Strengthening the motivation for the project

- Signature-based detection is not enough. To demonstrate so, we can use the example of Falco. Glossing over the nightmare that is installing it, Falco comes with a pre-set of rules that are fairly basic. For example, rules are triggered when SAs are created and deleted, secrets are accessed, and so on. However, these rules are exceedingly verbose and alerts are triggered for use cases that are not malicious at all. For example, Flannel renewing SA tokens, or the kube-controller-manager creating a secret for a SA when a namespace is created, are all examples of false positives that are triggered by Falco.
- ValidatingWebhookConfiguration is a powerful tool that can be used to validate requests to the API server. However, it is not used in the default installation of Kubernetes, and it is not easy to set up. Furthermore, it is not easy to understand what is happening when a request is denied, as the logs are not very verbose.

Behaviour analysis of the audit logs

Generic

- `watch` requests are long-running and are usually made by control plane components. Since the `ResponseStarted` and `ResponseComplete` events can be in different times, we chose to **remove** the `ResponseStarted` event from the logs.
 - Indeed, for us it is important to have the `requestReceivedTimestamp` that is present in all events anyway. With that, we can understand when the request was made and when it was completed.
 - `ResponseStarted` appears in `watch` requests, in `kubectl exec` requests, and in `delete` requests. In the latter case, `ResponseComplete` is sent once the object is actually deleted.
- `kubectl` requests are internally translated into a set of API calls that are made by the `kube-apiserver`. Further subsections detail such calls and how they can help us derive templates for the parser.

Namespaces

- Every time a namespace is created, the following things happen:
 - A `list` request is made to `/api/v1/namespaces/{}/resourcequotas` by the `kube-apiserver` for no apparent reason
 - * To add insult to injury, usually this line is out of order in the log
 - The `kube-controller-manager` gets the `service-account-controller` SA and generates a token for it
 - The SA `root-ca-publisher` puts the `root-ca.crt` in a ConfigMap in the namespace
 - The `service-account-controller` creates a SA called `default` in the namespace
- When a namespace is deleted, all subresources are deleted with a `deletecollection` request followed by a `list`. Exceptions include `Pods`, that weirdly are deleted by a `list`, then `deletecollection`, then `list` again. Also, when `configmaps` are deleted, the `kube-controller-manager` (and not the `root-ca-cert-publisher`) tries to create again the CM that was just deleted, but is rejected because the namespace is being terminated.

Other quirks

- When a secret is created for a SA the secret is instantly patched by the `kube-controller-manager`, which is in charge of actually generating that secret along with an object like

```
{"data":{"ca.crt":"...", "namespace":"...", "token":"..."}}
```

where the three fields are base64 encoded.

- Secrets can be traced back to the Pod they have been mounted into by either looking at the spec or monitoring for `watch` requests by the node that is hosting the pod.
- Users with access to `kubectl exec` can access the secrets of the pods they are exec-ing into using `printenv`.
- ResourceQuotas can be used for more than just limiting resources, they can also be used to limit the number of objects that can be created. This also applies to things like `serviceaccounts`: thus, it seems that every time a potentially limitable object is created, a `get` request is made to `/api/v1/{object}` to assess the current state.
- When CSRs are being approved or denied, the `update` verb is used and the actual decision is in the payload of the request. This makes it less straightforward to understand if the request was allowed or denied.
- When old-style ServiceAccount tokens are used, instead of the new CSR style Users, the API server detects the *offending* action and adds a label `kubernetes.io/legacy-token-last-used` to the Secret.

Impersonation

- Users with access to the `impersonate` verb and one of `users`, `groups`, and `serviceaccounts` can impersonate other identities.
- This is correctly reflected in the audit logs, but could be a problem implementation-wise.

Control plane components

- Control plane components' "background noise" has been filtered out and the details are available in queries.org.

Future work

- Using KWOK
- Ask for a master's student to create a simulator with KWOK, defining scenarios and gathering the audit log